

Task 02 - Part 02: Answers (Items 2 & 3)

C# Fundamentals - Compilation Model & Type Casting

2. What's the difference between compiled and interpreted languages, and where does C# fit in?

Compiled languages are translated directly from source code into machine code (or another low-level form) by a compiler *before* the program runs. The result is a standalone executable that the CPU can run directly, which generally makes compiled programs fast to execute, since there's no translation step at runtime. The trade-off is that compiled code is usually tied to a specific platform/OS, and any change to the source requires recompiling. Examples: C, C++.

Interpreted languages are translated and executed line-by-line (or statement-by-statement) at runtime by an interpreter, without producing a separate standalone executable ahead of time. This makes them more flexible and portable (the same source can run on any machine with the right interpreter installed) and easier to test quickly, but generally slower than compiled code because translation happens while the program runs. Examples: Python, JavaScript (in its classic form).

Where C# fits in: C# uses a **hybrid** approach. The C# compiler (Roslyn) first compiles the source code into an intermediate, platform-independent bytecode called **CIL / MSIL** (Common Intermediate Language), producing a .dll or .exe assembly. This is the "compiled" step. Then, at runtime, the **.NET CLR (Common Language Runtime)** uses a **JIT (Just-In-Time) compiler** to translate that intermediate code into native machine code for the specific machine it's running on, right before it executes. So C# is compiled twice: once ahead-of-time into IL (portable, like an interpreted language's portability), and once more at runtime from IL into native code (giving near-compiled performance). This is why C# is often described as a **managed, compiled language** rather than purely compiled or purely interpreted.

3. Compare between implicit, explicit, Convert, and Parse casting.

C# offers four main ways to convert a value from one type to another, each suited to a different situation:

Method	How it works	When it's used	Risk of data loss / errors
Implicit casting	Happens automatically; no special syntax needed. The compiler performs the conversion on its own.	Widening conversions where no data can be lost, e.g. int to double, or a derived class to its base class.	None - always safe, which is why the compiler allows it silently.

Method	How it works	When it's used	Risk of data loss / errors
Explicit casting	Uses cast syntax (Type)value, written manually by the developer.	Narrowing conversions between related/compatible numeric types or object types, e.g. double to int, or object back to a specific class.	Possible - can truncate decimals or throw an <code>InvalidCastException</code> if the types aren't actually compatible at runtime.
Convert class	Static methods such as <code>Convert.ToInt32()</code> , <code>Convert.ToDouble()</code> , <code>Convert.ToString()</code> , etc.	Converting between unrelated types (e.g. string to int, or handling null values, since <code>Convert.ToInt32(null)</code> returns 0 instead of throwing).	Throws <code>FormatException</code> if the string isn't a valid number, or <code>OverflowException</code> if the value is out of range - but handles null gracefully.
Parse / TryParse	Static methods on the target type itself, e.g. <code>int.Parse()</code> , <code>double.Parse()</code> , or the safer <code>int.TryParse()</code> which returns a bool instead of throwing.	Specifically converting a string into a numeric or other value type, most often for user input.	<code>Parse</code> throws <code>FormatException</code> on invalid input and <code>ArgumentNullException</code> on null. <code>TryParse</code> avoids exceptions entirely by returning false on failure.

In short: implicit is automatic and safe; explicit is manual and can lose data; Convert is flexible and null-safe but works across broader type ranges; Parse (and TryParse) is specialized for turning strings into value types, with TryParse being the safest option when handling user input.